

Build your own coding agent

Outline

- Introduction
- Large Language Models
- OpenAI Completions API
- Tool Calling
- Coding Agents

Introduction

Goals:

- Form a technical *intuition*
- **Build your own agent**

Non-goals:

- Teaching you to use agents effectively
- Train/finetune your own LLM
- Discuss ethics/merits of LLMs
- Discuss company AI strategy

Large Language Models (LLMs)

- *Stateless* magic black box
 - `def LLM(input: str) -> str`
- Internal representation: [tokens](#) (demo)
 - [Andrey Karpathy: Deep Dive into LLMs like ChatGPT](#)
- Context window: maximum `input` length

LLM Providers

-  OpenAI (ChatGPT/Codex)
-  Anthropic (Claude)
-  Google (Gemini)
-  Mistral AI
-  Z.ai/Moonshot/MiniMax

Reseller: [OpenRouter](#)

OpenAI API: Endpoints

Two endpoints:

1. `/v1/chat/completions`
 - Older, widely-supported (OpenRouter)
 - Stateless
2. `/v1/responses`
 - Newer, more features, worse support
 - Optional state: `previous_response_id`

Anthropic API: `/v1/messages` (Claude Code)

OpenAI API: Messages Request

```
POST http://127.0.0.1:1234/v1/chat/completions HTTP/1.1
Content-Type: application/json
Authorization: Bearer <YOUR_API_KEY>
```

```
{
  "model": "openai/gpt-oss-20b",
  "messages": [
    { "role": "system", "content": "You answer in one word." },
    { "role": "user", "content": "Which country invented brie?" }
  ]
}
```

OpenAI API: Messages Response

```
{
  "id": "chatcmpl-14g3zft88igr1q1vr3crc2q",
  "object": "chat.completion",
  "model": "openai/gpt-oss-20b",
  "choices": [
    {
      "message": {
        "role": "assistant",
        "content": "France",
        "reasoning": "Need one word answer. Brie invented in France. So answer: France.",
        "tool_calls": []
      },
      "finish_reason": "stop"
    }
  ]
}
```


OpenAI API: Messages Demo

- `demos/chat-turn1.http`
- `demos/chat-turn2.http`

Tool Calling: History

'Historically' (2023) tool calling worked using *few-shot prompting*: **demo!**

- Downsides?

Tool Calling: OpenAI API

```
{
  "messages": [ ... ],
  "tools": [
    {
      "type": "function",
      "function": {
        "name": "get_weather",
        "description": "Get the current weather for a location",
        "parameters": {
          "type": "object",
          "required": [
            "location"
          ],
          "properties": {
            "location": {
              "type": "string",
              "description": "The location to get the weather for"
            }
          }
        }
      }
    }
  ]
}
```

Tool Calling: Demo

- `demos/tools-turn1.http`
- `demos/tools-turn2.http`

Coding Agents: Definition

An LLM agent runs tools in a loop to achieve a goal.

- Simon Willison ([September 2025](#))

Coding Agents: Pseudocode

```
messages = [{ "role": "system", "content": SYSTEM_PROMPT }]
while True:
    user_input = input()
    messages.append({ "role": "user", "content": user_input })

    while True:
        response = llm_request({
            "model": "openai/gpt-oss-20b",
            "messages": messages,
            "tools": TOOL_DEFINITIONS,
        })["choices"][0]["message"]
        messages.append({ "role": "assistant", **response })

        if not response["tool_calls"]:
            print(response["content"])
            break

    tool_result: str = call_tool(response)
    messages.append({ "role": "tool", "content": tool_result, "tool_call_id": id })
```

Coding Agents: Caching

Possible to modify the `messages` between turns, but **more expensive**.

Model providers use caching to speed up requests with the same `messages` prefix, otherwise every token has to be re-processed.

Caching



Cache Invalidation



Templates

- Python/C++
- Functionality
 - JSON POST (with logging)
 - `bash_command`
 - `Pi's system-prompt.md`

Coding Time!

Repository: <https://github.com/mrexodia/agent>